

CSE220 Signals and Linear Systems

Offline on Convolution

Building and Applying a Discrete-Time LTI System

1. Introduction

In this offline, you will implement reusable classes for finite discrete-time signals and discrete-time linear time-invariant (LTI) systems. An LTI system is completely characterized by its impulse response $h[n]$, and its output for an input $x[n]$ is

$$y[n] = \sum_{k=-\infty}^{\infty} x[k]h[n-k].$$

You will compute this output in two equivalent ways:

1. **Superposition of responses:** form the component responses $x[k]h[n-k]$ and add them.
2. **Sliding multiply-and-add:** compute each output value directly from the convolution sum.

The two results must be verified to be identical. The same implementation will then process input files containing either a custom impulse response or all built-in impulse responses supplied in the template code.

Part I: Implementation

2. Class DiscreteSignal

The `DiscreteSignal` class represents a finite discrete-time signal over the inclusive integer range

$$n_{\min} \leq n \leq n_{\max}.$$

Values outside the stored range are treated as zero whenever a signal value is requested.

Constructor

```
DiscreteSignal(start_time, end_time)
```

Creates a zero-valued signal over the inclusive range from `start_time` to `end_time`.

Required Methods

`get_value_at_time(t)` Returns the signal value at time t ; returns zero when t is outside the stored range.

set_value_at_time(t, value) Sets the value at time t . The index must lie within the stored range.

shift(k) Returns a new signal representing $x[n - k]$. Positive k shifts the signal to the right, and negative k shifts it to the left.

add(other) Returns the sum of two signals, correctly handling different time ranges.

multiply(scalar) Returns a new signal representing a scalar multiple of the original signal.

plot(title, save_path=None) Produces a stem plot over the signal's stored range and saves it when a path is provided.

The class must store the signal's start time, end time, and values. The exact internal representation and any additional helper methods are your choice.

3. Class LTISystem

The `LTISystem` class represents a discrete-time LTI system using an arbitrary finite impulse response $h[n]$.

Constructor

```
LTISystem(impulse_response)
```

Initializes the system using a `DiscreteSignal` representing $h[n]$.

Output Range

If $x[n]$ is stored over $[n_{x,\min}, n_{x,\max}]$ and $h[n]$ is stored over $[n_{h,\min}, n_{h,\max}]$, the complete output must use the range

$$n_{x,\min} + n_{h,\min} \leq n \leq n_{x,\max} + n_{h,\max}.$$

This range must be determined automatically.

Required Methods

get_response_components(input_signal) Returns the nonzero component responses $x[k]h[n - k]$ associated with the input samples.

output_by_superposition(input_signal) Computes the complete output by adding all component responses returned by `get_response_components`.

get_contributions_at_time(input_signal, n) Returns the nonzero terms $x[k]h[n - k]$ that contribute to the selected output value $y[n]$.

output_at_time(input_signal, n) Returns the scalar output value $y[n]$ using the convolution sum.

output(input_signal) Computes the complete output by evaluating the convolution sum separately at every output index.

The two complete-output methods must be independent: **output_by_superposition** must use component-response superposition, while **output** must use the sliding multiply-and-add calculation.

Part II: Verification

4. Required Verification for Every Impulse Response

For every custom or built-in impulse response, **main.py** must compute

```
y_superposition = system.output_by_superposition(input_signal)
y_convolution    = system.output(input_signal)
```

Compare the two outputs over their full range and calculate

$$\max_n |y_{\text{sup}}[n] - y_{\text{conv}}[n]|.$$

The maximum difference must be zero up to normal floating-point tolerance. Built-in convolution or correlation functions must not be used for either output.

For every processed impulse response, **report.txt** must include the input values, impulse-response values, output values, output range, maximum absolute difference, and paths of the saved figures. A built-in run must contain a separate report section for every built-in impulse response.

Part III: File-Based Convolution Application

5. Input Files

main.py must read the problem from a text file supplied through the command line. The submission must include four examples:

- **inputs/1.txt**: small signal with a custom impulse response;
- **inputs/2.txt**: the same small signal with **built-in**;
- **inputs/3.txt**: long signal with a custom impulse response;
- **inputs/4.txt**: the same long signal with **built-in**.

Input Signal

The first two lines contain the input time range and values:

```
input_start input_end
x[input_start] x[input_start+1] ... x[input_end]
```

The number of values must equal **input_end - input_start + 1**. The signal must not be assumed to start at zero.

Impulse-Response Mode

Line 3 contains either

```
custom
```

or

```
built-in
```

The spelling `builtin` may also be accepted.

For `custom`, lines 4–5 contain the impulse-response range and values:

```
impulse_start impulse_end  
h[impulse_start] h[impulse_start+1] ... h[impulse_end]
```

For `built-in`, the program must apply all six impulse responses supplied in the template code, in the given order:

1. identity:

$$h[0] = 1$$

2. 3-point moving average:

$$h[0] = h[1] = h[2] = \frac{1}{3}$$

3. 5-point moving average:

$$h[0] = h[1] = h[2] = h[3] = h[4] = \frac{1}{5}$$

4. 7-point moving average:

$$h[0] = h[1] = h[2] = h[3] = h[4] = h[5] = h[6] = \frac{1}{7}$$

5. weighted smoothing:

$$h[0] = 0.5, \quad h[1] = 0.3, \quad h[2] = 0.2$$

6. first difference:

$$h[0] = 1, \quad h[1] = -1$$

Do not change their definitions, names, or order. Each built-in system must be represented as an ordinary `DiscreteSignal` and processed through `LTISystem`.

Sample Custom Input

```
-2 3  
1 0 2 -1 0 3  
custom  
-1 2  
1 2 0 -1
```

Sample Built-In Input

```
-2 3  
1 0 2 -1 0 3  
built-in
```

6. Required Visualizations

Every processed impulse response must produce both of the following:

1. a stem-plot figure containing $x[n]$, $h[n]$, and $y[n]$;
2. a grayscale color-block figure containing only $x[n]$ and $y[n]$.

For the color-block figure, normalize each displayed signal separately to the range 0–255 and show it as one horizontal row of adjacent grayscale blocks. Keep the horizontal-axis labels readable for long signals.

7. Running the Program and Saving Results

The project root must contain `cmd.txt`, listing the exact commands used for the four provided input files. For example:

```
python3 main.py inputs/1.txt --out-dir outputs/1  
python3 main.py inputs/2.txt --out-dir outputs/2  
python3 main.py inputs/3.txt --out-dir outputs/3  
python3 main.py inputs/4.txt --out-dir outputs/4
```

`main.py` must accept the argument

```
--out-dir <directory>
```

and create the selected directory when necessary. Each output directory must have the form

```
<out-dir>/  
|-- report.txt  
|-- plot/  
'-- color/
```

For a custom input, save

```
plot/convolution.png  
color/convolution.png
```

For a built-in input, save one stem plot and one color plot for every built-in impulse response using these filenames:

```
plot/builtin_1_identity.png  
color/builtin_1_identity.png  
plot/builtin_2_moving_average_3.png  
color/builtin_2_moving_average_3.png
```

```
plot/builtin_3_moving_average_5.png
color/builtin_3_moving_average_5.png
plot/builtin_4_moving_average_7.png
color/builtin_4_moving_average_7.png
plot/builtin_5_weighted_smoothing.png
color/builtin_5_weighted_smoothing.png
plot/builtin_6_first_difference.png
color/builtin_6_first_difference.png
```

Existing files in the selected output directory may be overwritten.

8. Required Program Structure

signal_lti.py

Contains the complete `DiscreteSignal` and `LTISystem` classes. It must not contain input-file-specific or filter-specific processing logic.

main.py

Reads the command-line arguments and input file, processes custom or built-in impulse responses, performs verification, generates the report and figures, and saves all results in the selected output directory.

Use the standard structure:

```
def main():
    # Required tasks
    pass

if __name__ == "__main__":
    main()
```

The class implementations must not be duplicated inside `main.py`.

9. Restrictions and General Requirements

- Do not use built-in convolution or correlation operations, including `numpy.convolve`, `scipy.signal.convolve`, `scipy.signal.fftconvolve`, or similar functions.
- You may use NumPy for ordinary numerical operations, Matplotlib for plotting, and the Python standard library.
- Correctly handle negative indices, different signal ranges, asymmetric impulse responses, and automatically determined output ranges.
- Signal operations must return new objects where appropriate and must not rely on hardcoded signal lengths or output values.
- The class code must remain general and reusable; built-in-filter-specific logic belongs

outside the two classes.

10. Mark Distribution

Component	Marks
DiscreteSignal representation, indexing, and constructor	8
Signal shifting	6
Signal addition and scalar multiplication	7
Signal plotting and time-range handling	4
DiscreteSignal Total	25
Response-component generation	12
Output by superposition	10
Contributions and output at one time	10
Complete convolution-sum output and output range	13
LTISystem Total	45
Verification, report, and generated visualizations	10
File reading and custom/built-in processing	9
Stem and color visualizations	6
Code organization, readability, and documentation	5
Total	100

11. Submission

Create and zip a folder named with your student ID:

```
<student_id>/
|-- inputs/
|   |-- 1.txt
|   |-- 2.txt
|   |-- 3.txt
|   '-- 4.txt
|-- cmd.txt
|-- signal_lti.py
'-- main.py
```

Submit it as `<student_id>.zip`. Do not include generated outputs, images, virtual environments, cache folders, or unrelated files.

Deadline: Friday, 24 July, 11:59pm